

ABC454 D - (xx) 题解

题意概括

给定两个只由 (、x、) 组成的字符串 A 和 B。

可以进行两种操作：

- 把一个子串 (xx) 变成 xx
- 把一个子串 xx 变成 (xx)

问是否能把 A 变成 B。

解题思路

这题的关键是找到一个不会随操作改变的规范形。

观察

如果我们不断执行：

(xx) $\rightarrow$ xx

那么无论什么时候把一段 xx 变成 (xx)，最后在继续做删除时，这一对括号迟早还是会被删掉。

因此，一个字符串真正重要的不是中途加了哪些这种“可消掉的括号”，而是把所有能消掉的 (xx) 都删完以后，最后剩下什么。

于是定义一个规范化过程：

- 反复把字符串中的字面量子串 (xx) 替换成 xx
- 直到字符串里不再出现 (xx) 为止

记最终结果为 $\text{normalize}(S)$ 。

那么：

- 一次 $xx \rightarrow (xx)$ 操作不会改变 $\text{normalize}(S)$ ；
- 一次 $(xx) \rightarrow xx$ 操作也不会改变 $\text{normalize}(S)$ 。

所以如果 A 能变成 B，就必然有：

$\text{normalize}(A) = \text{normalize}(B)$

反过来，如果两个串规范化后相同，那么：

- A 可以通过若干次 $(xx) \rightarrow xx$ 变成 $\text{normalize}(A)$;
- B 也可以通过若干次 $(xx) \rightarrow xx$ 变成 $\text{normalize}(B)$;
- 若二者相同, 就说明 A 和 B 都能变到同一个串, 因此也一定可以互相变换。

于是答案就是判断:

$\text{normalize}(A) == \text{normalize}(B)$

如何线性求规范形

用一个栈按顺序读入字符。

每加入一个字符后, 检查栈顶四个字符是否正好是:

(xx)

如果是, 就把它删掉并改成:

xx

这个替换可能又和前面的字符形成新的 (xx) , 所以继续检查即可。

每个字符只会进栈、出栈常数次数, 因此整体是线性的。

正确性说明

记 $N(S)$ 为把字符串 S 中所有可删除的 (xx) 全部删完后得到的结果。

先证明它是操作不变量:

- 若执行 $(xx) \rightarrow xx$, 只是直接做了一次规范化中的一步, 显然 $N(S)$ 不变;
- 若执行 $xx \rightarrow (xx)$, 新加上的这一对括号正是规范化时可以立刻删掉的部分, 所以 $N(S)$ 也不变。

因此, 任何可达变换都不会改变 $N(S)$ 。

再证明充分性:

- 对任意字符串 S , 总可以通过不断做 $(xx) \rightarrow xx$ 变到 $N(S)$;
- 如果 $N(A) = N(B)$, 那么 A 和 B 都能变到同一个字符串, 所以它们互相可达。

因此, A 能变成 B 当且仅当 $N(A) = N(B)$ 。程序正是按这个条件判断, 所以算法正确。

复杂度

设单个测试用例字符串总长度为 L 。

- 时间复杂度: $O(L)$

- 空间复杂度: $O(L)$

参考实现

```
#include<bits/stdc++.h>
using namespace std;

string normalize(const string &s){

    string st;
    for(char ch : s){
        st.push_back(ch);

        while((int)st.size() >= 4){
            int n = (int)st.size();
            if(st[n - 4] == '(' && st[n - 3] == 'x' && st[n - 2] == 'x' && st[n - 1] == ')'){
                st.pop_back();
                st.pop_back();
                st.pop_back();
                st.pop_back();
                st.push_back('x');
                st.push_back('x');
            } else {
                break;
            }
        }
    }

    return st;
}

int main(){

    int T;
    cin >> T;

    while(T--){
        string a, b;
        cin >> a >> b;
```

```
if(normalize(a) == normalize(b)){  
    cout << "Yes\n";  
} else {  
    cout << "No\n";  
}  
}  
  
return 0;  
}
```